PROGRAMMING & CS FUNDAMENTALS

# Linked Lists Visual Guide

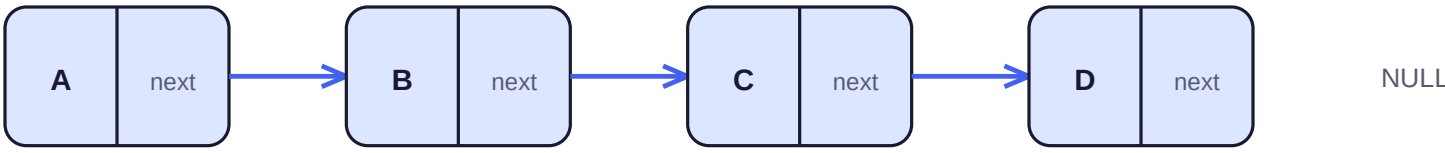
Nodes and pointers —
O(1) surgery at the cost of O(n) reach.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/6

# Anatomy of a Node

Data plus a pointer to the next node — the list is a chain of handshakes.




Each node: [ data | next* ]. The list only stores a head pointer — reach node k by hopping k times: O(n).

```
struct Node { int data; Node* next; };
Node* head -> [A|*] -> [B|*] -> [C|*] -> [D|*] -> NULL
```

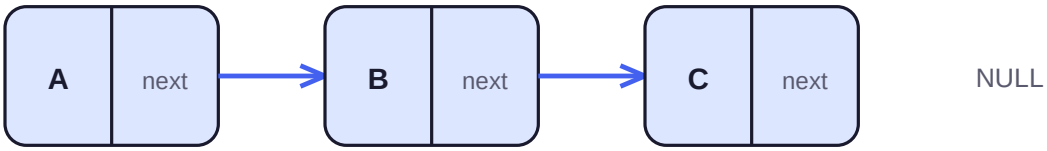
No contiguous block needed: nodes scatter across the heap. Growing the list never copies — but you lose cache locality.

STEP 2/6

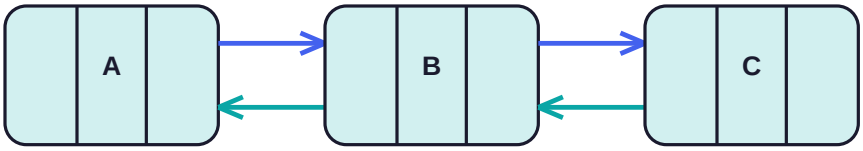
# Singly vs Doubly Linked

One direction is cheap; two directions cost a pointer per node.

### Singly: forward only




### Doubly: forward + back (prev pointer)



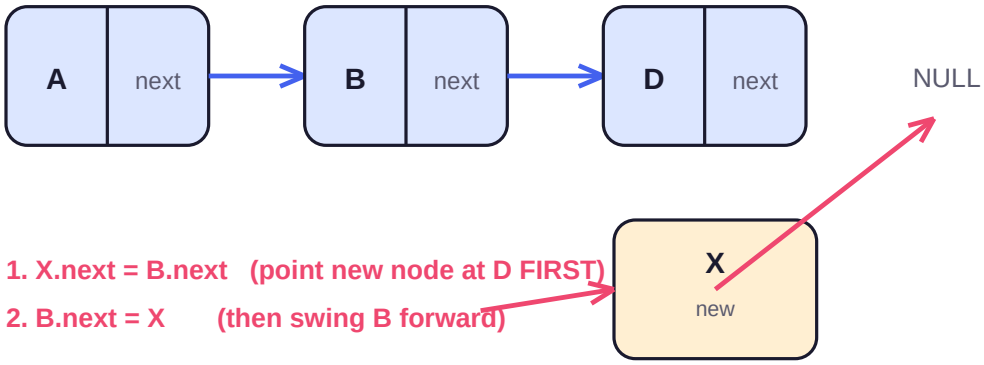

- Doubly: O(1) delete given the node (no predecessor hunt); singly needs the previous node.
- Cost: one extra pointer per node + more careful surgery (two links to rewire).
- Circular variants: tail.next = head — handy for round-robin schedulers.

STEP 3/6

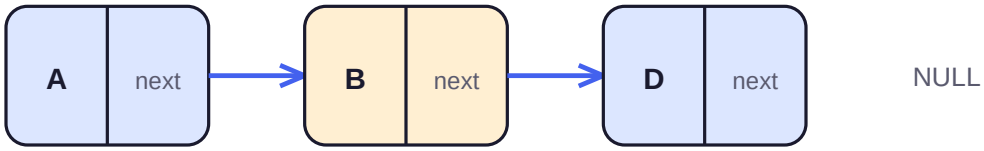
# Pointer Surgery: Insert & Delete

Rewire in the right order — or lose the rest of the list.

**Insert X after B:**




**Delete C (have its predecessor B):**



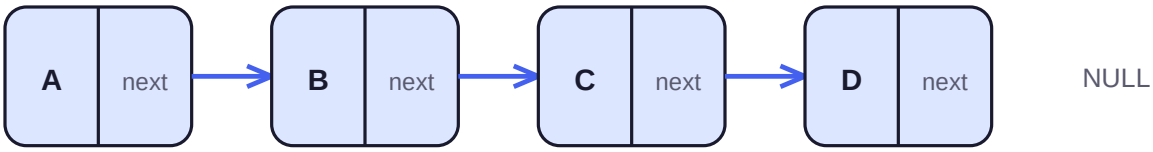

B.next = C.next — C is unlinked in one assignment: O(1). Then free(C).

Order matters: doing step 2 first orphans D forever (memory leak). New link before breaking the old one — always.

STEP 4/6

# Reversal: The Classic

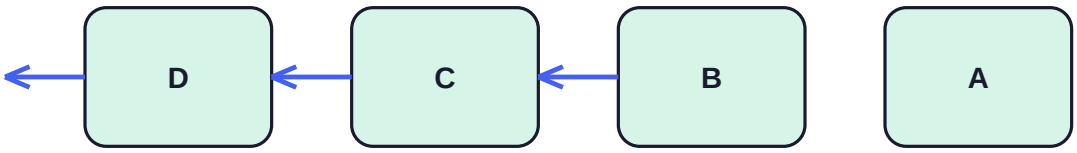
Three pointers walk the list, flipping links as they go.




Pointers: prev=NULL, curr=head. Each step:

```
while curr:
    nxt = curr.next   # 1. save next (else we lose it)
    curr.next = prev  # 2. flip the link backwards
    prev, curr = curr, nxt   # 3. advance both
head = prev           # new head = old tail
```

**Result:**

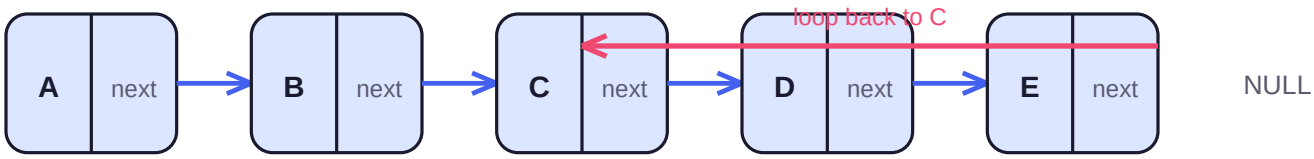



O(n) time, O(1) space — the iterative version beats recursion here (no stack depth).

STEP 5/6

# Floyd's Cycle Detection

Tortoise and hare: if there's a loop, the fast runner laps the slow one.




```
slow = fast = head
while fast and fast.next:
    slow = slow.next        # 1 step
    fast = fast.next.next  # 2 steps
    if slow is fast: return True   # met inside the loop
return False
```

- Why it works: in a cycle, fast gains one node per round — a meeting is inevitable.
- O(n) time, O(1) space — no hash set needed.
- Bonus: from the meeting point, classic math finds the cycle's start (mu).

STEP 6/6

# Linked List vs Array

Pick by access pattern, not by habit.

**Random access arr[i]**

Array O(1) vs List O(n)

**Insert/delete at ends**

Array: back O(1) vs List: both O(1)*

**Insert/delete middle**

Array O(n) shifts vs List O(1) rewire*

**Memory overhead**

none per element vs 1-2 pointers/elem

**Cache locality**

excellent vs poor (scattered)

*List O(1) assumes you already hold the node/position — reaching it still costs O(n). Arrays win by default; lists win for constant middle surgery.

STEP ✓I✓

# Key Takeaways

## Linked Lists — Remember

- Nodes hold data + next (doubly: + prev); the list is a chain of handshakes, O(n) to reach node k.
- Insert/delete is O(1) pointer surgery — but rewire new-links-before-old or you leak the tail.
- Iterative reversal: save next, flip link, advance — O(n) time, O(1) space.
- Floyd's tortoise-and-hare finds cycles in O(n) time, O(1) space.
- Default to arrays; reach for lists when middle insert/delete dominates and locality doesn't matter.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev